

Assignment-I

Pratik Beniwal

05-06-2026

Part I: Short Answer Questions

Q1: A normal file system is just like saving things in different folders on your computer, where we have to do all the work to connect them. A DBMS is a smart software that does this for us, keeping everything organized and safe for many users at once. For example, if the power goes out while transferring money between bank accounts, a normal file system might lose that money. A DBMS prevents this because it has a built-in safety net that guarantees the transfer either fully finishes or resets completely.

Q2: A strong entity can stand on its own and has its own unique ID, like a student with a student ID. A weak entity cannot exist without a strong one, like a student's emergency contact, and doesn't have a full ID of its own. To give a weak entity its own ID, we combine the ID of the strong entity with a small piece of the weak entity's data. The connection between them is called an identifying relationship.

Q3: It should be modeled as a multivalued attribute. A simple attribute only holds one thing, like an age, and a composite attribute is one thing made of smaller parts, like an address. since a student can have several different phone numbers at the same time, we need a multivalued attribute, which is specially designed to hold a list of items for one single person.

Q4: A super key is any combination of columns that can find one specific row. A candidate key is the smallest possible super key, meaning if we take away any column from it, it won't work anymore. For the Student table, the candidate keys are the roll number and the email, because both can uniquely find a student on their own. Super keys that are not candidate keys could be the combination of roll number and name, or email and branch. These still work to find a student, but they include extra useless info to get the job done.

Q5: The foreign key constraint is violated here. This rule stops us from adding fake references. For example, it makes sure we can't enroll a student in a class if that student doesn't even exist in the main database. Because this rule is broken, the database will instantly block the action and show an error.

Q6: To connect a many-to-many relationship, you have to make a brand-new table, often called a bridge or junction table. Its primary key is made by combining the primary keys from both of the original tables. We can't just use a normal column because basic

database rules say we cannot stuff a list of things into a single box. If a student takes five courses, we can't just write all five in one column, and adding separate blank columns for each course is just bad design.

Part II: Long Answer Question

Part A – Entity and Relationship Identification

Entities and Key Attributes:

- **Customer:** key attribute is Customer_ID.
- **Restaurant:** key attribute is Restaurant_ID.
- **Order:** key attribute is Order_ID.
- **Menu Item:** key attribute is Item_ID.

Relationships:

- **Customer to Order (1:N):** One customer can place many orders, but a specific order is tied to only one customer.
- **Order to Restaurant (N:1):** Many orders can be sent to the same restaurant, but a single order is made at exactly one restaurant.
- **Order to MenuItem (M:N):** One order can have many different items, and the exact same item can be part of many different people's orders.
- **Restaurant to MenuItem (1:N):** One restaurant offers many items on its menu, but a specific menu item listing belongs to that one restaurant.

Part B – Attribute Classification

- **Multivalued Attribute:** A customer's phone numbers.
explanation: This qualifies because a single customer might have a home phone and a mobile phone saved on their profile at the same time.
- **Composite Attribute:** A customer's delivery address.
explanation: This qualifies because a full address can be broken down into smaller, distinct pieces like street name, city, and zip code.

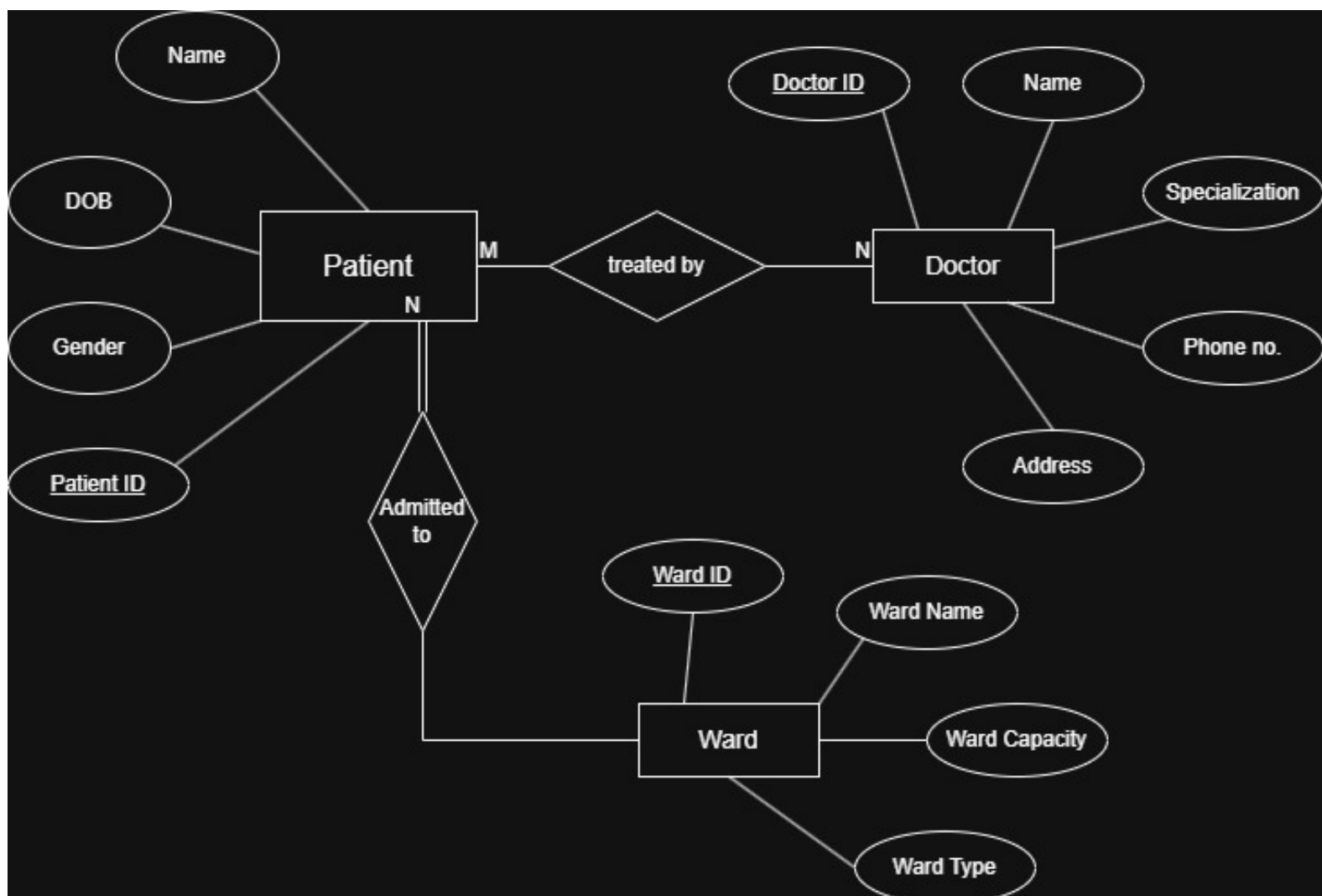
Part C – Relational Schema Design

Schema Notation:

Order_Details(order_id, item_id, quantity)

- **Primary Key:** The combination of {order_id, item_id}.
- **Foreign Keys:** order_id (connects back to the main Order table) and item_id (connects back to the main MenuItem table).
- **Additional attribute:** quantity.
explanation: This is needed because if we order x quantity, the database needs a specific place to store that number for this exact order.

Part III : ER Diagram Design



Question 2: Design Justification

- **Doctor-Patient Relationship (M:N):** The relationship labeled “treated by” is many-to-many because a single doctor treats multiple patients, and a patient can be seen by different doctors. In a relational database, this creates an associative entity, such as a “Consultation” table. Its composite primary key would be made by combining the Doctor ID and the Patient ID, likely alongside a timestamp so every single visit is uniquely recorded.
- **Participation Rules:**
 - **Total Participation:** The diagram explicitly shows a double line between the Patient entity and the “Admitted to” relationship. This means every single patient must be assigned to a hospital ward to exist in this system.
 - **Partial Participation:** The Doctor entity has partial participation in the “treated by” relationship. This makes sense because a newly hired doctor can exist in the hospital’s database without currently having any patients assigned to them.
- **Attribute Design Decision:** I modeled the doctor’s “Address” as a simple attribute (shown as a single oval) instead of a composite one. Breaking it down into street, city, and zip code wasn’t necessary for this specific design. Storing it as a single basic record keeps the database simple and efficient, as we likely won’t need to run complex location-based searches on the doctors.